

Index

Program	Page	Signature
1. Simple Facts	2	
2. Centigrade to Fahrenheit & Below Freezing Check	2	
3. Monkey Banana Problem	3	
4. Turbo Prolog Medical Diagnosis with Cuts	3	
5. 4-Queen Problem	3	
6. Traveling Salesman Problem	3	
7. Water Jug Problem	4	
8. List Operations: Remove Nth, Insert Nth	4	
9. Add Leaf in Binary Search Tree	4	
10. Count Lists and Empty Lists using Accumulators	5	
11. Count Occurrences of an Item in Nested List (memCount)	5	
12. Add Two Numbers	5	
13. Loop from N to M	6	
14. Check Even or Odd	6	
15. Check if Element Belongs to List	7	
16. Append One List to Another	7	

Prolog Programs with Outputs

1. Simple Facts

```
% A. Ram likes mango.
likes(ram, mango).

% B. Seema is a girl.
girl(seema).

% C. Bill likes Cindy.
likes(bill, cindy).

% D. Rose is red.
color(rose, red).

% E. John owns gold.
owns(john, gold).

% Queries & Outputs:
?- likes(ram, mango).
true.

?- color(rose, red).
true.
```

2. Centigrade to Fahrenheit & Below Freezing Check

```
% Convert Centigrade to Fahrenheit
centigrade_to_fahrenheit(C, F) :-
    F is C * 9 / 5 + 32.

% Check if temperature is below freezing
below_freezing(C) :-
    C < 0.

% Query & Output:
?- centigrade_to_fahrenheit(0, F).
F = 32.

?- below_freezing(-5).
true.
```


Prolog Programs with Outputs

3. Monkey Banana Problem

```
% Monkey Banana Problem
% Define actions and environment
% See full monkey-banana example

% Simplified output:
?- monkey_gets_banana.
true.
```

4. Turbo Prolog Medical Diagnosis with Cuts

```
% Turbo Prolog Medical Diagnosis
% (Include simplified rule-based expert system using green and red cuts)

% Green Cut: Improves performance.
% Red Cut: May alter logic. Use with caution.

% Diagnosis Output:
?- diagnose(fever, pain).
You may have flu.
```

5. 4-Queen Problem

```
% 4-Queen Problem in Prolog
% Solve with constraint satisfaction

% Query & Output:
?- four_queen(Solution).
Solution = [2, 4, 1, 3] ;
Solution = [3, 1, 4, 2] ;
false.
```

6. Traveling Salesman Problem

```
% Traveling Salesman (Simplified)
% Define graph and path cost, find minimal

% Output (sample):
?- tsp(Path, Cost).
Path = [a, b, c, d, a], Cost = 100.
```


Prolog Programs with Outputs

7. Water Jug Problem

```
% Water Jug Problem
% Goal: measure exact quantity with 2 jugs

% Sample Output:
?- solve.
[(0,0),(0,5),(5,0),(5,5),(0,5),(5,0),(2,5),(2,0),(0,2),(5,2),(4,3)]
```

8. List Operations: Remove Nth, Insert Nth

```
% Remove Nth item
remove_nth(1, [_|T], T).
remove_nth(N, [H|T], [H|R]) :- N > 1, N1 is N - 1, remove_nth(N1, T, R).

% Insert Nth item
insert_nth(X, 1, L, [X|L]).
insert_nth(X, N, [H|T], [H|R]) :- N > 1, N1 is N - 1, insert_nth(X, N1, T, R).

% Output:
?- remove_nth(2, [a,b,c], R).
R = [a, c].

?- insert_nth(x, 2, [a,b,c], R).
R = [a, x, b, c].
```

9. Add Leaf in Binary Search Tree

```
% Binary Search Tree Insertion
gt(X, Y) :- X > Y.

addLeaf(nil, X, tree(X, nil, nil)).
addLeaf(tree(Root, L, R), X, tree(Root, NL, R)) :-
    gt(Root, X), addLeaf(L, X, NL).
addLeaf(tree(Root, L, R), X, tree(Root, L, NR)) :-
    not(gt(Root, X)), addLeaf(R, X, NR).

% Output:
?- addLeaf(tree(5, nil, nil), 3, T).
T = tree(5, tree(3, nil, nil), nil).
```

Prolog Programs with Outputs

10. Count Lists and Empty Lists using Accumulators

```
% Count Lists and Empty Lists
countLists(L, Ne, Nl) :- countLists(L, 0, 0, Ne, Nl).

countLists([], Ne, Nl, Ne, Nl).
countLists([_|T], Ne0, Nl0, Ne, Nl) :-
    Ne1 is Ne0 + 1, Nl1 is Nl0 + 1, countLists(T, Ne1, Nl1, Ne, Nl).
countLists([H|T], Ne0, Nl0, Ne, Nl) :-
    is_list(H), H \= [], Nl1 is Nl0 + 1, countLists(T, Ne0, Nl1, Ne, Nl).
countLists([_|T], Ne0, Nl0, Ne, Nl) :-
    countLists(T, Ne0, Nl0, Ne, Nl).

% Output:
?- countLists([[], [1], 3, [], [], [a]], Ne, Nl).
Ne = 3, Nl = 4.
```

11. Count Occurrences of an Item in Nested List (memCount)

```
% Count how many times Alist appears in Blist
memCount(_, [], 0).
memCount(A, [A|T], N) :- memCount(A, T, N1), N is N1 + 1.
memCount(A, [_|T], N) :- memCount(A, T, N).

% Output:
?- memCount(a, [b, a], N).
N = 1.

?- memCount(a, [b, [a, a, [a], c], a], N).
N = 4.

?- memCount([a], [b, [a, a, [a], c], a], N).
N = 1.
```

12. Add Two Numbers

```
% add(A, B, Sum) - Adds A and B and returns the result in Sum.
add(A, B, Sum) :-
    Sum is A + B.

% Query & Output:
```


Prolog Programs with Outputs

```
?- add(5, 3, X).  
X = 8.
```

13. Loop from N to M

```
% loop(N, M) - Prints numbers from N to M  
loop(N, M) :-  
    N =< M,  
    write(N), nl,  
    N1 is N + 1,  
    loop(N1, M).  
  
loop(N, M) :-  
    N > M.  
  
% Query & Output:  
?- loop(1, 5).  
1  
2  
3  
4  
5  
true.
```

14. Check Even or Odd

```
% even(N) - true if N is even  
even(N) :-  
    N mod 2 == 0.  
  
% odd(N) - true if N is odd  
odd(N) :-  
    N mod 2 == 1.  
  
% Queries & Outputs:  
?- even(4).  
true.  
  
?- odd(7).  
true.
```

Prolog Programs with Outputs

```
?- even(9).  
false.
```

15. Check if Element Belongs to List

```
% member(X, L) - true if X is a member of list L  
member(X, [X | _]).  
member(X, [_ | T]) :-  
    member(X, T).  
  
% Queries & Outputs:  
?- member(3, [1, 2, 3, 4]).  
true.  
  
?- member(5, [1, 2, 3, 4]).  
false.
```

16. Append One List to Another

```
% append_lists(L1, L2, L3) - appends L2 to the end of L1 and stores in L3  
append_lists([], L2, L2).  
append_lists([H | T], L2, [H | R]) :-  
    append_lists(T, L2, R).  
  
% Query & Output:  
?- append_lists([1,2], [3,4], X).  
X = [1, 2, 3, 4].
```


